

The four main features of OOPs are as follows:-

1. Encapsulation :-

Encapsulation is the process of binding the data member and member function into a single unit , that single unit is called class.

Example:-

```
class Student {  
    private String name;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

2. Inheritance :-

Inheritance allows to inherit the property and behaviour of the parent class by using a child class.

Example(Single inheritance):-

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("The dog barks.");  
    }  
}
```

Example(Multiple Inheritance):-

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats food");  
    }  
}
```

```

}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy weeps");
    }
}
public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat(); // From Animal
        p.bark(); // From Dog
        p.weep(); // From Puppy
    }
}

```

3. Polymorphism:-

Polymorphism allows a method to behave differently based on the object which is calling it.

Example:-

```

class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}

```

4. Abstraction:-

Abstraction hides unnecessary data and shows only the essential data.

Example:-

```

abstract class Vehicle {

```

```
        abstract void move();
    }

    class Car extends Vehicle {
        void move() {
            System.out.println("Car is moving");
        }
    }
}
```

Inheritance in OOPs:-

Single Inheritance:-

```
class A {
    void display() {
        System.out.println("Class A");
    }
}

class B extends A {
    void show() {
        System.out.println("Class B");
    }
}

public class Main {
    public static void main(String[] args) {
        B obj = new B();
        obj.display(); // From class A
        obj.show();    // From class B
    }
}
```

Multilevel Inheritance:-

```
class A {
```

```
void display() {  
    System.out.println("Class A");  
}  
}
```

```
class B extends A {  
    void show() {  
        System.out.println("Class B");  
    }  
}
```

```
class C extends B {  
    void print() {  
        System.out.println("Class C");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
        obj.show();  
        obj.print();  
    }  
}
```

Difference between the Method Overloading and Method Overriding:-

Method Overloading:-

Multiple methods in the same class with the same name but different parameters.

Example:

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

Method Overriding:-

A subclass provides a specific implementation of a method already defined in the parent class.

Example:-

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Encapsulation:-

Encapsulation improves code maintainability, security, and reusability. It prevents direct access to class fields and provides controlled access via methods.

Example:-

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0)  
            balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```